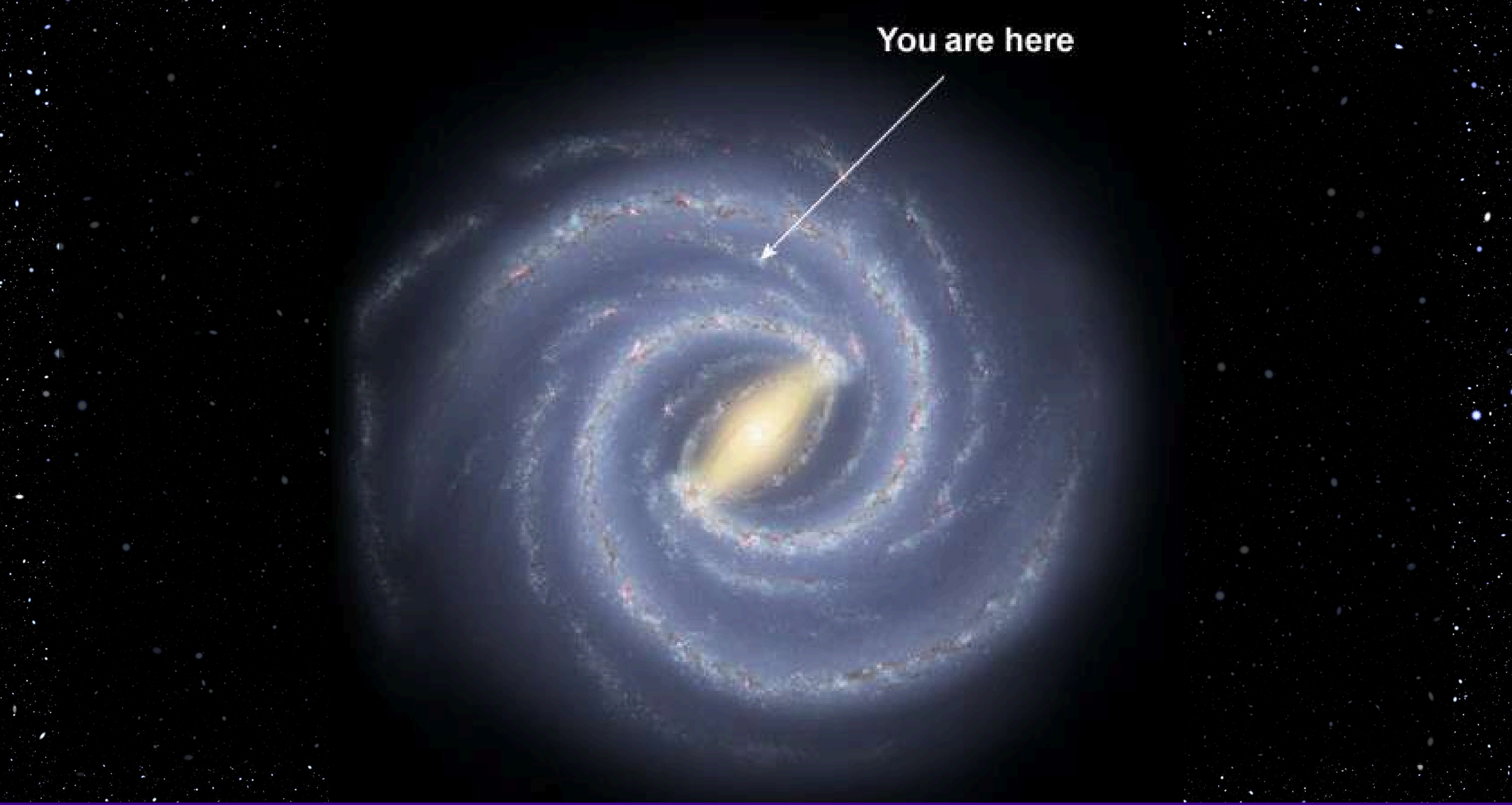
Making Gravity SWIFTer: GPU offloads for gravity in the SWIFT cosmology code

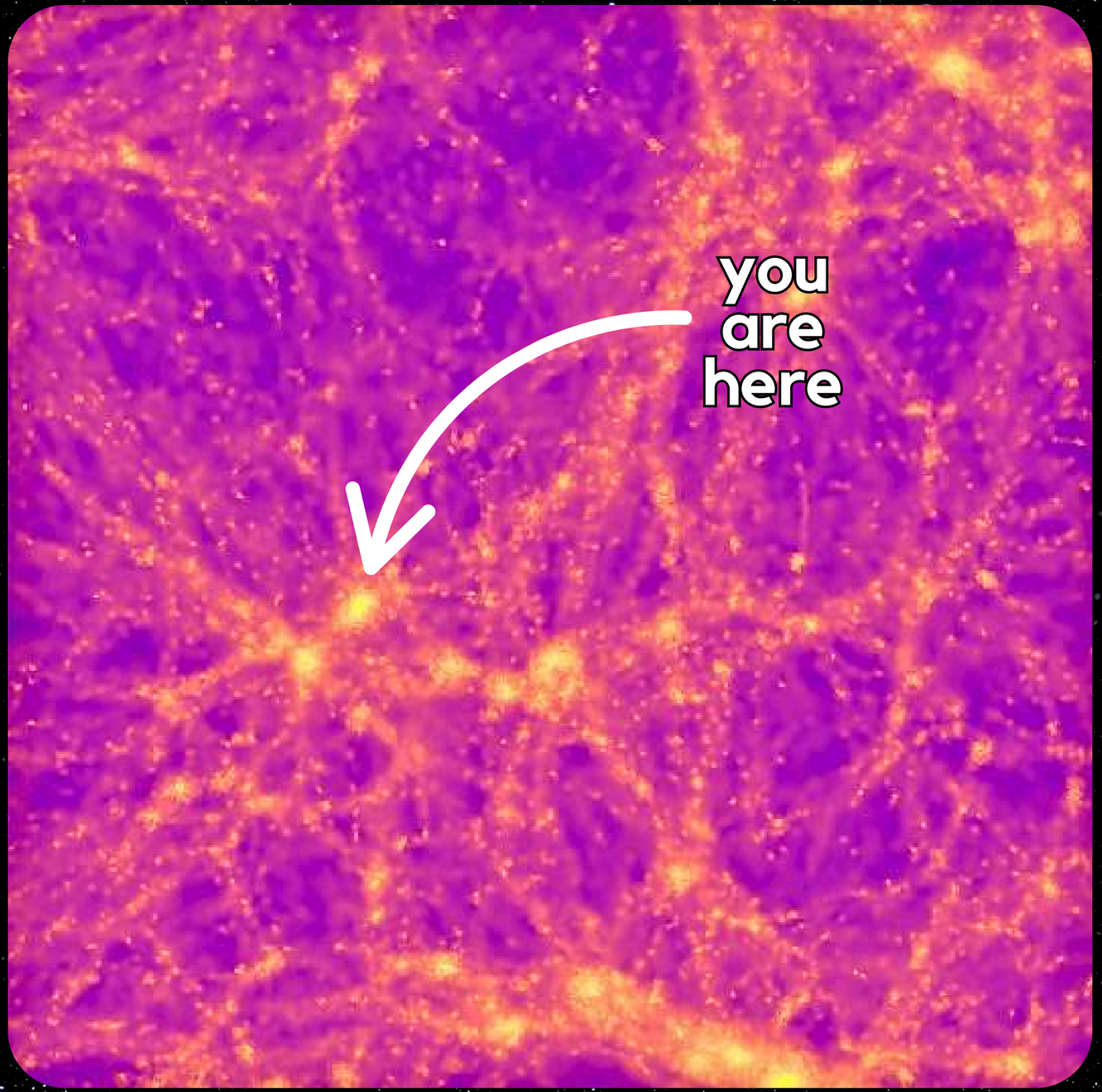
Sarah Johnston

(she/they) - 4th Year PhD Student

Supervisors: Alastair Basden, Sownak Bose, Carlton Baugh

sarah.c.johnston@durham.ac.uk


You are here

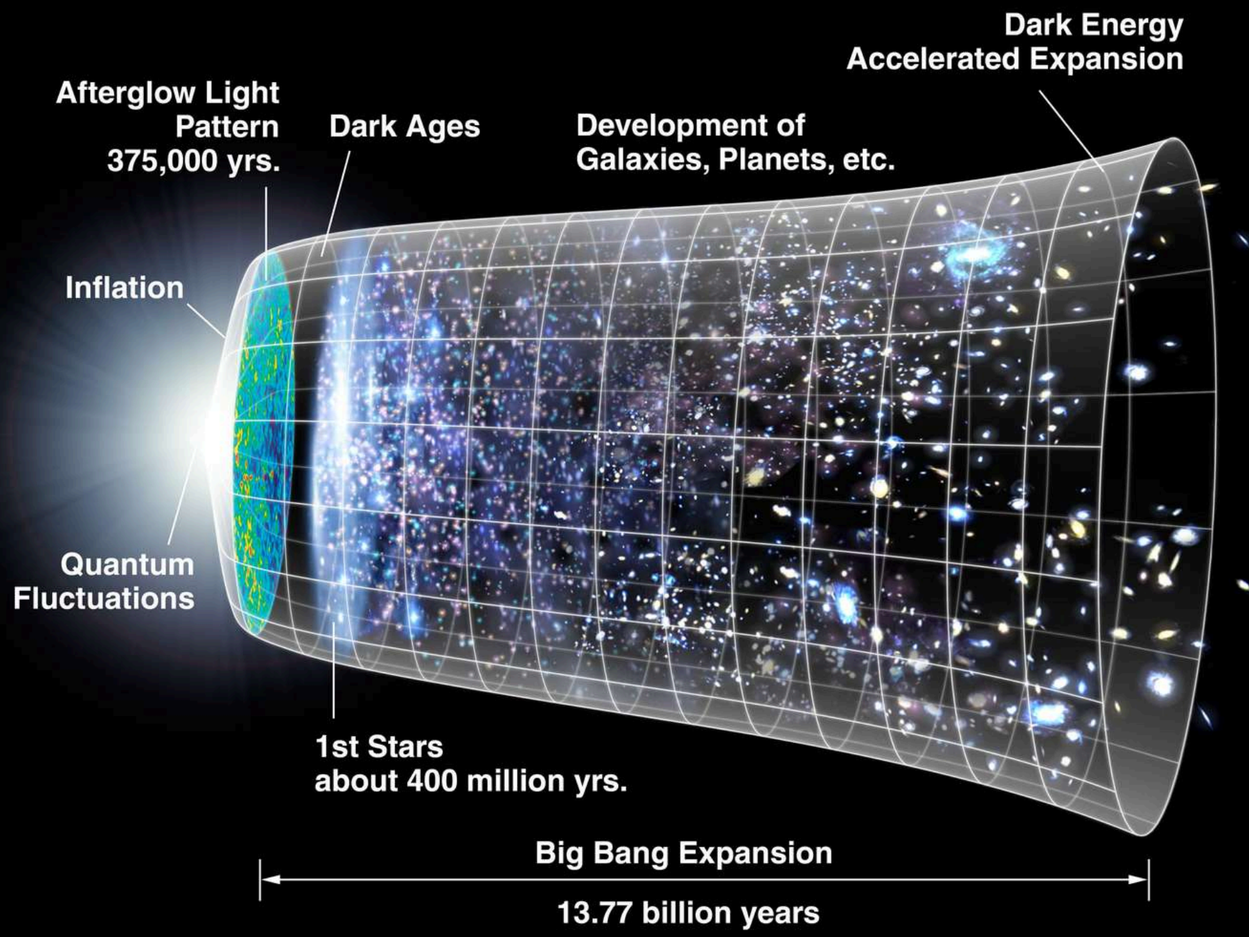

you
are
here

COSMOLOGY

Cosmology is the study of the origin, formation, and nature of the universe

We investigate how structures form and evolve e.g. galaxies

We need large, memory-intensive simulations



HPC & GPUS

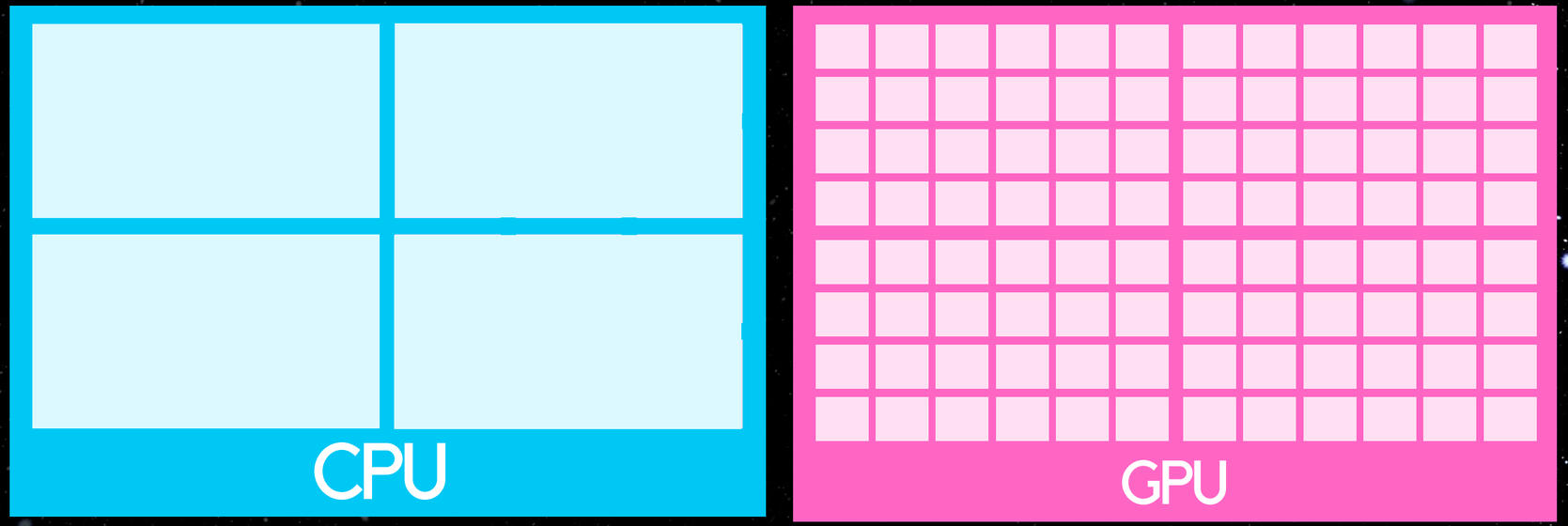
HPC systems allow to run these large simulations

New systems trending towards using GPUs for performance, sustainability etc.

GPUS

Graphics processing units are multi-cored accelerators which allow for multiple calculations to be carried out simultaneously

The GPU cores are less powerful, but that allows for more cores to be placed on a chip



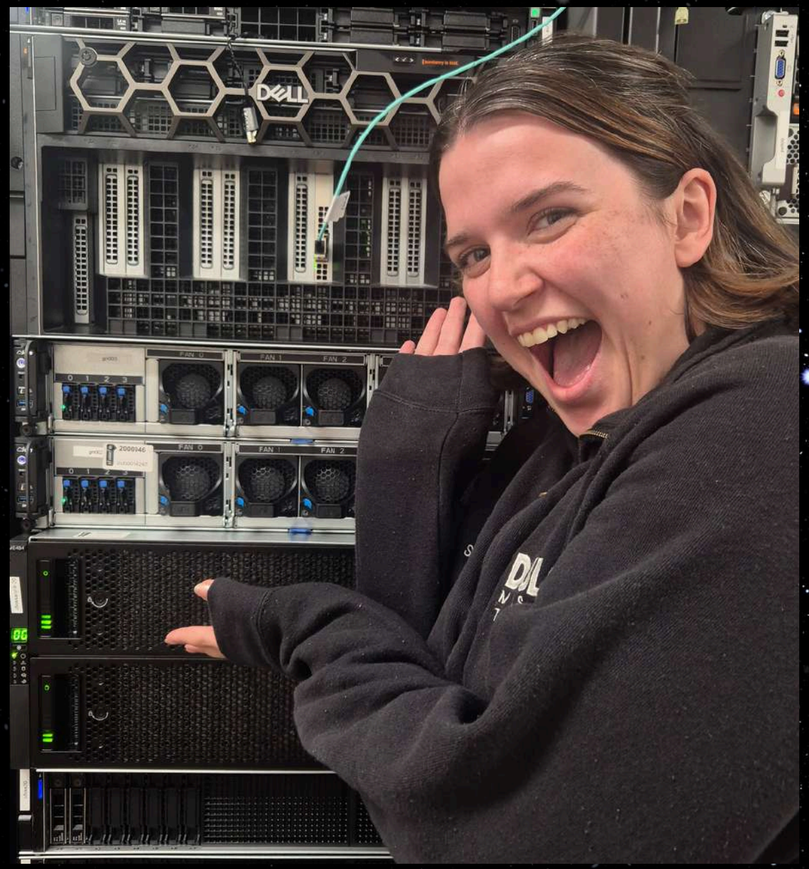
WHY GPUS

Motivation:

- faster runtimes
- power efficiency and sustainability
- 'future-proof' codes for heterogeneous architectures

Benefits:

- more FLOPs per Joule than a CPU
- high levels of user control over parallelisation
- specific coding features e.g. streams, thread-block level control, hidden memory transfers



(IDEAL) CODING FOR GPUS

GPUs are great for:

- operations that are easy to distribute (no inter-dependencies)
- repetitive operations
- large amounts of compute
- things that don't need lots of memory

COSMOLOGY CODES IN HPC

Cosmology codes often require large amounts of memory

They are generally well-suited or designed specifically for CPU only systems

Lots of work across the field is going into porting these codes to be GPU compatible

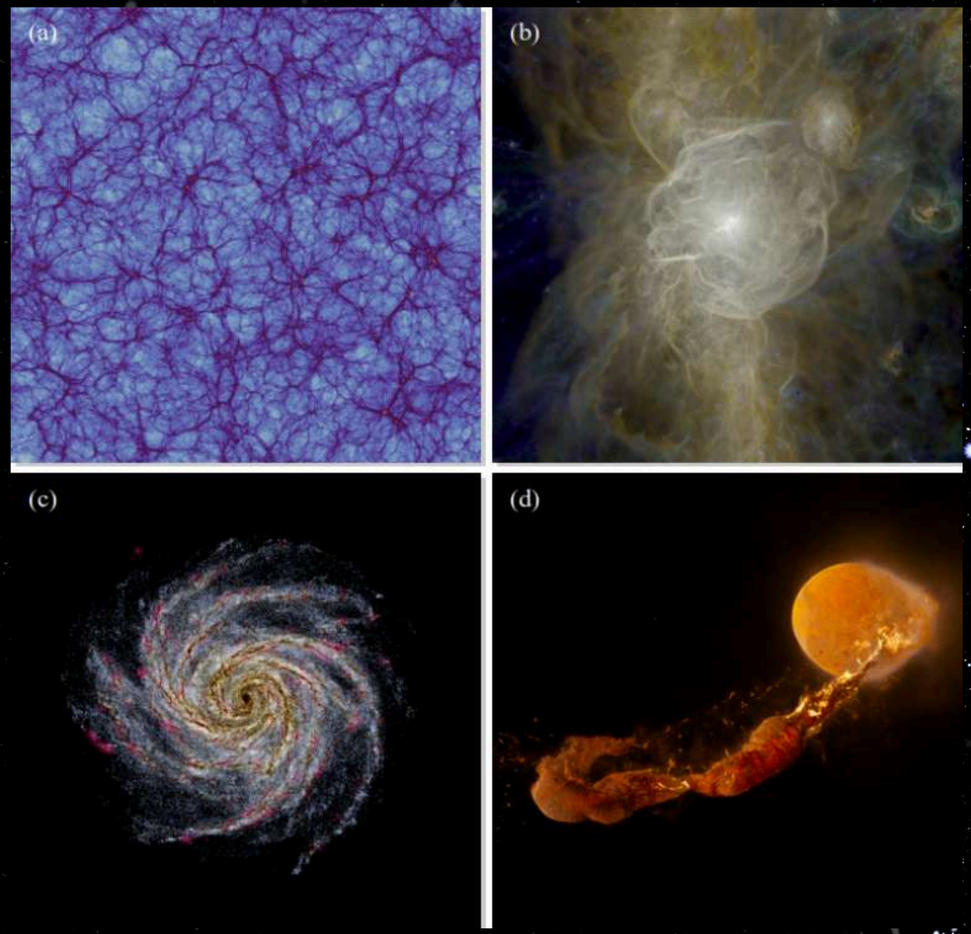
SWIFT

SWIFT is a smoothed particle hydrodynamics (SPH) cosmology code (Schaller et al. 2023)

SWIFT is based upon:

- task-based parallelism
- graph-based domain decomposition
- communications

GPU need to fit into this framework



GPU VERSION
VS
GPU OFFLOAD

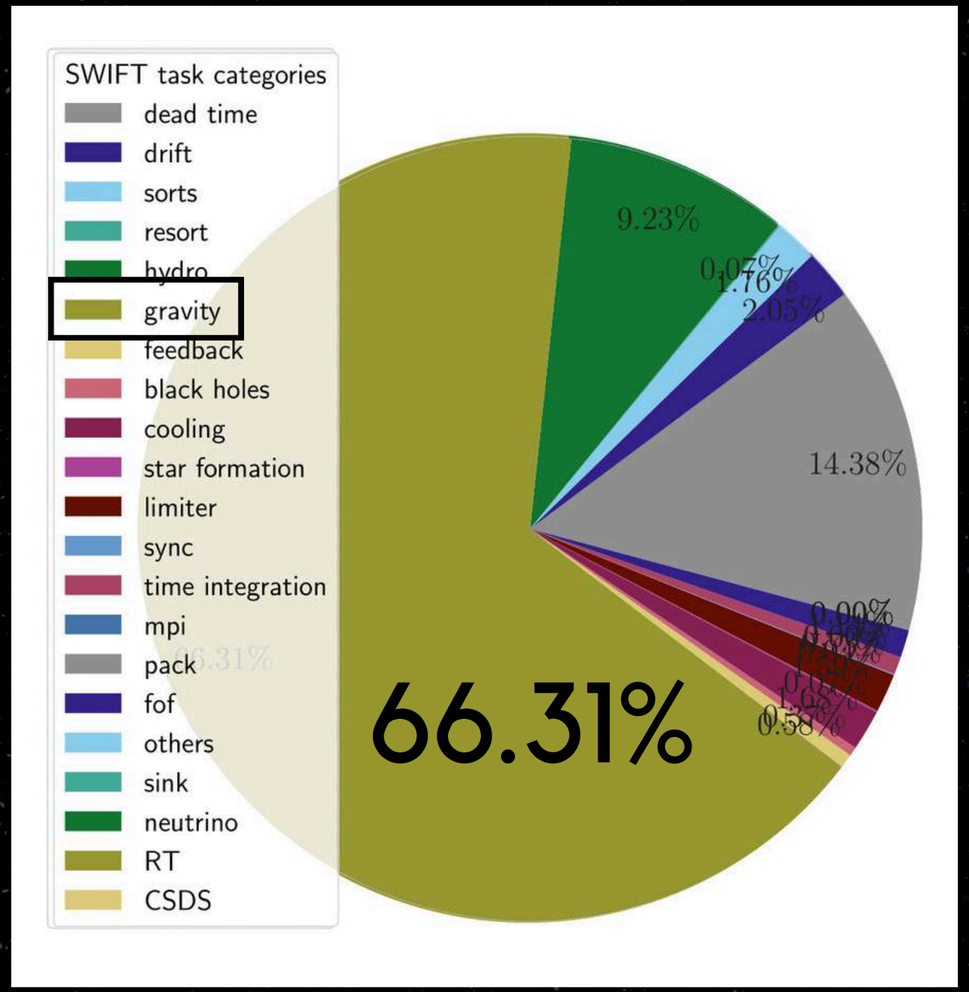
SWIFT GRAVITY

Large amounts of time spent in gravity

Gravity utilised in all SWIFT runs

Gravity task types:

- self (within a cell)
- pair (between neighbouring cells)



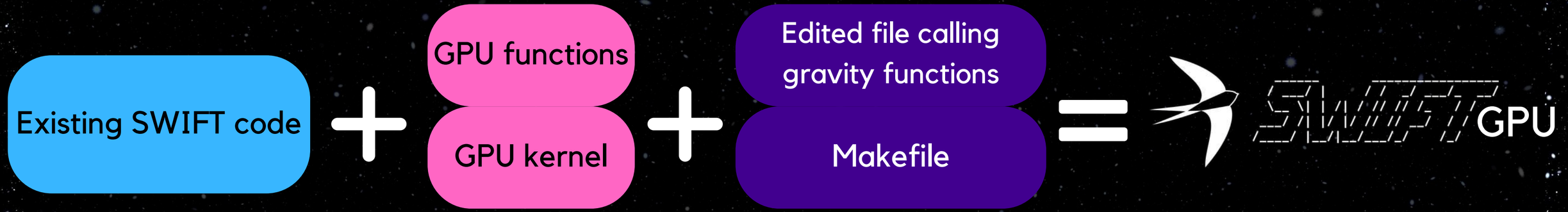
EAGLE run to z = 0

AIMS

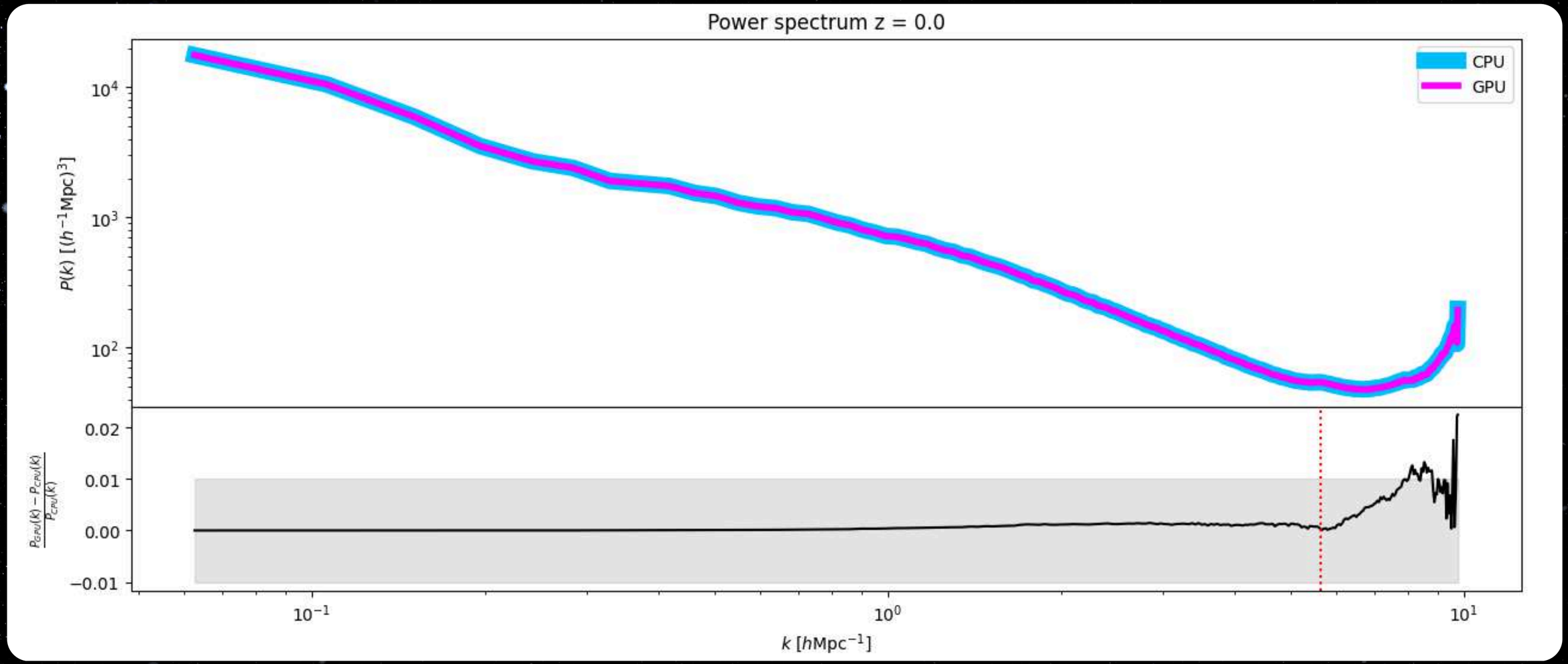
- GPU offload (a section of) the gravity calculations
- Minimally invasive (minimal changes to code, GPU option rather than seperate GPU code)
- Proof of concept vs long-term solution

METHOD

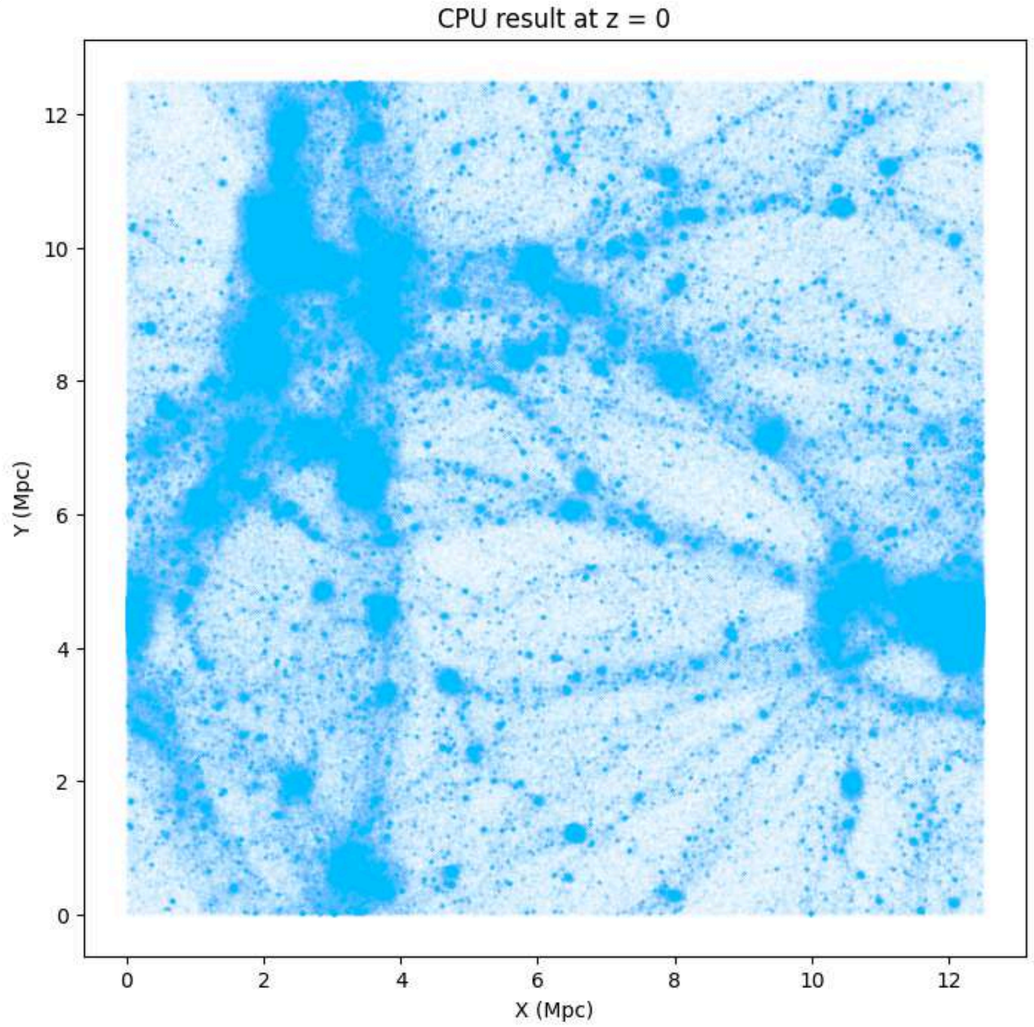
- Use the existing SWIFT code as a base
- Write GPU kernel to replace individual functions in CUDA
- Call GPU kernel in place of usual CPU function in C code
- Use Makefile to link CPU and GPU code

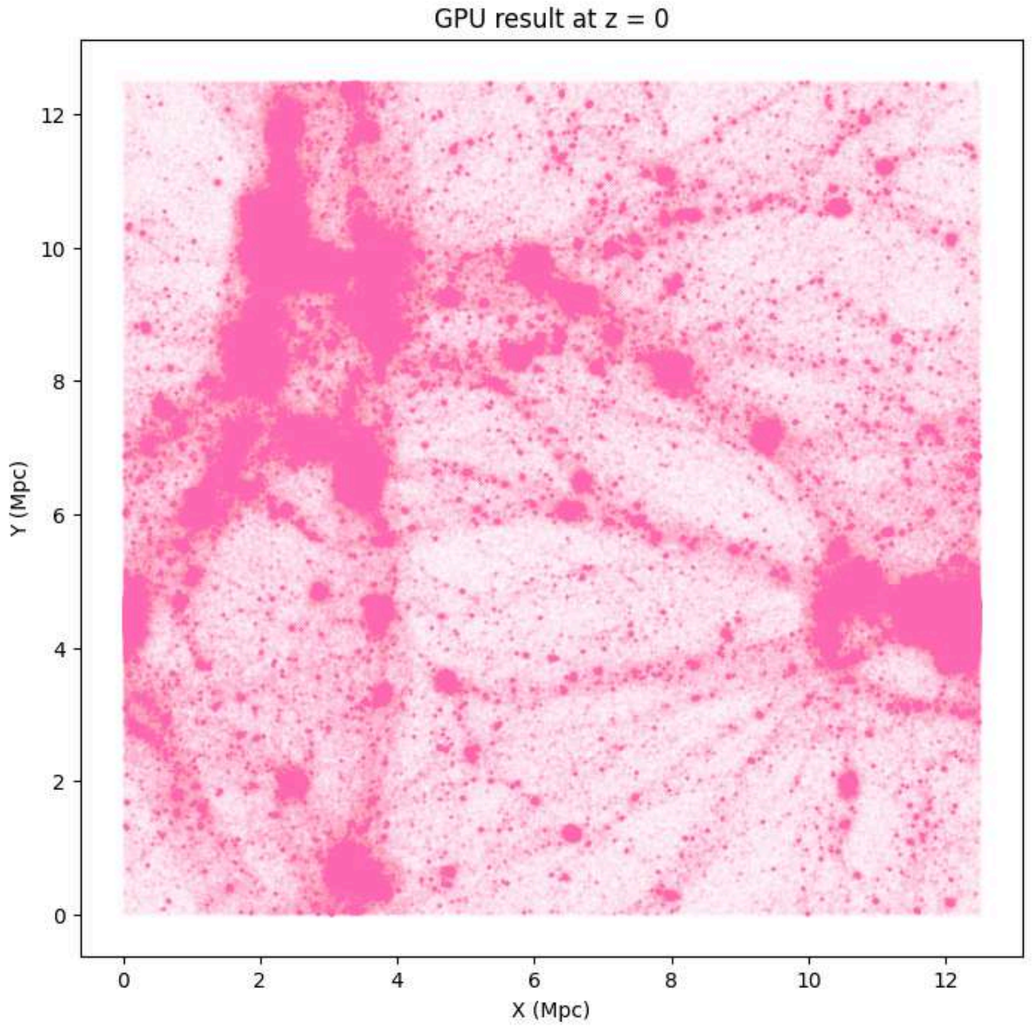


ACCURACY RESULTS



ACCURACY RESULTS

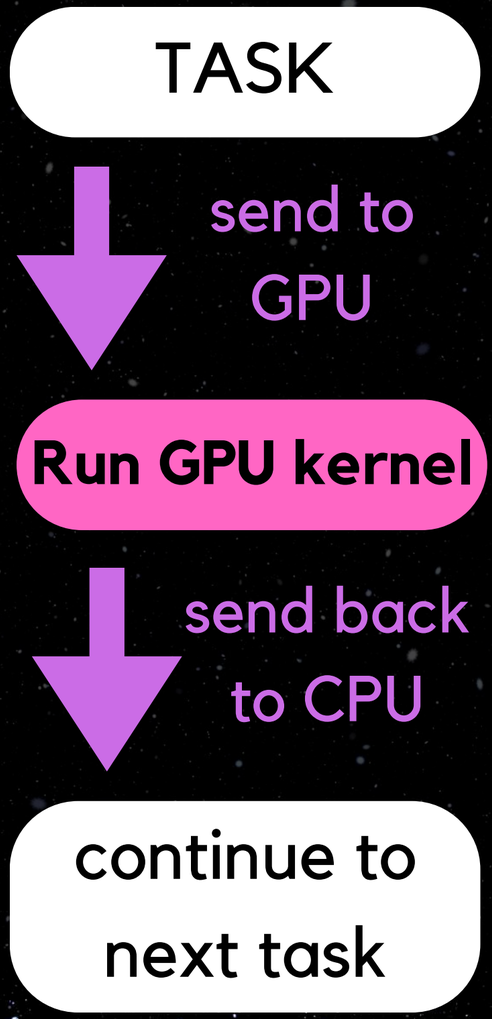




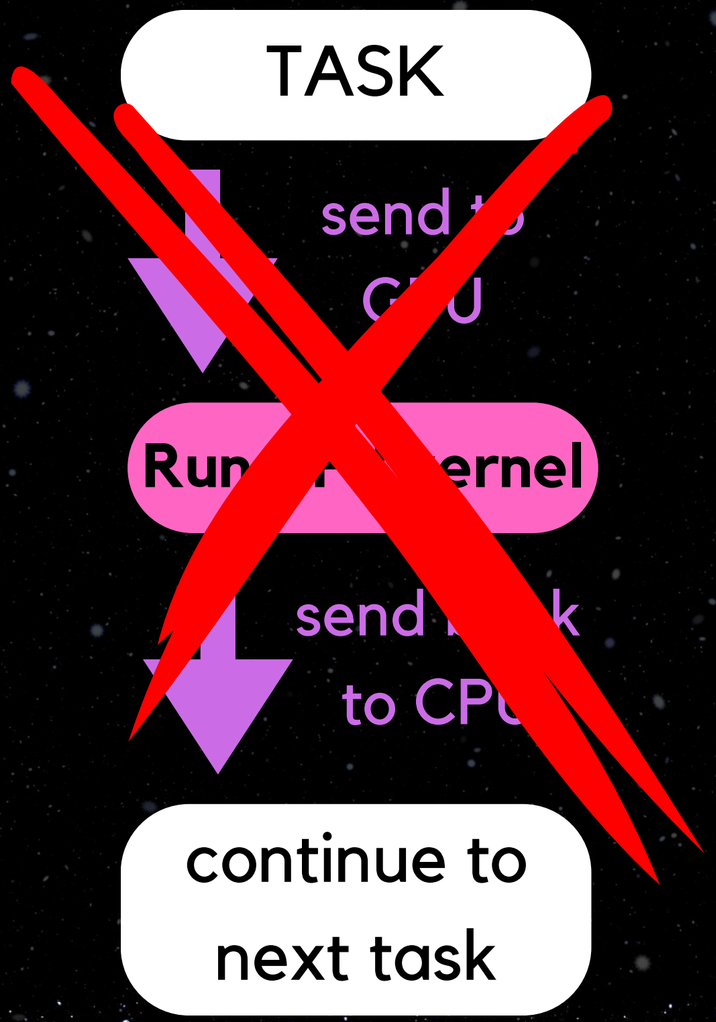
THE PROBLEM

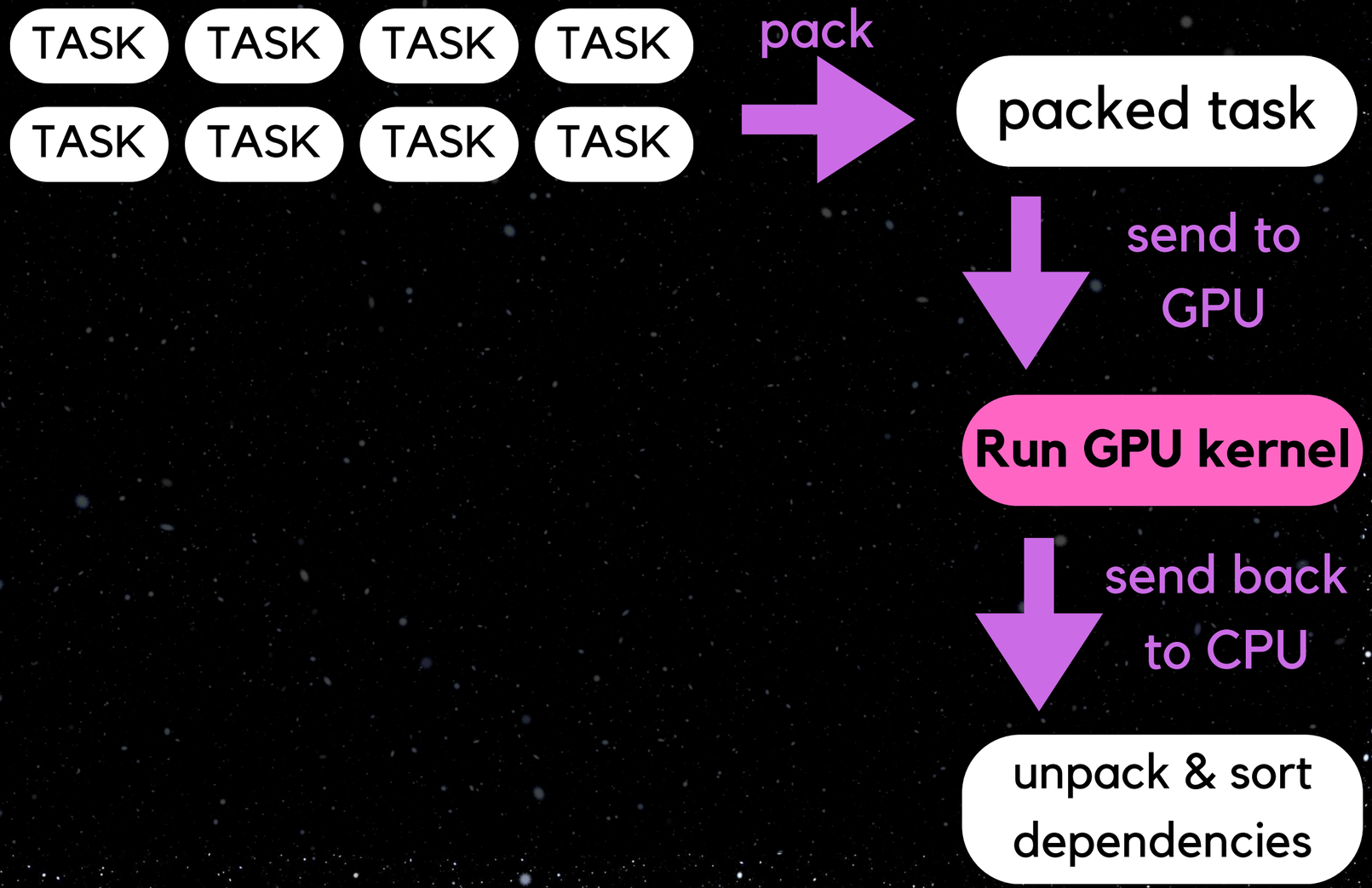
- Not enough work for the GPU with one cell at a time
- Memory transfer time far longer than calculation
- Solution → send more than one cell at once so cells can be reused multiple times per transfer for self and pair operations

TASK BUNDLING



TASK BUNDLING





DEALING WITH TASKS

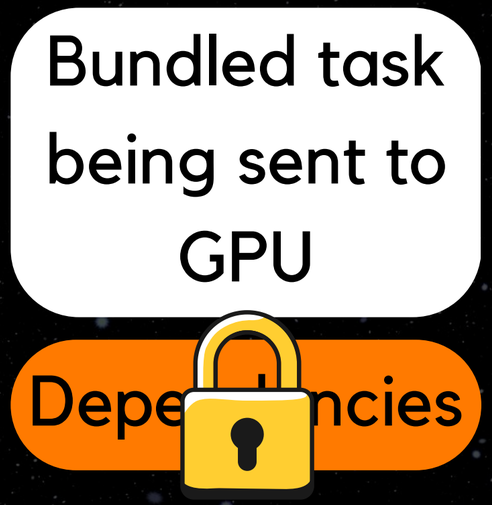
- Lots of task dependencies in SWIFT → need to ensure any tasks are still enqueued correctly afterwards

Bundled task being sent to GPU

Dependencies

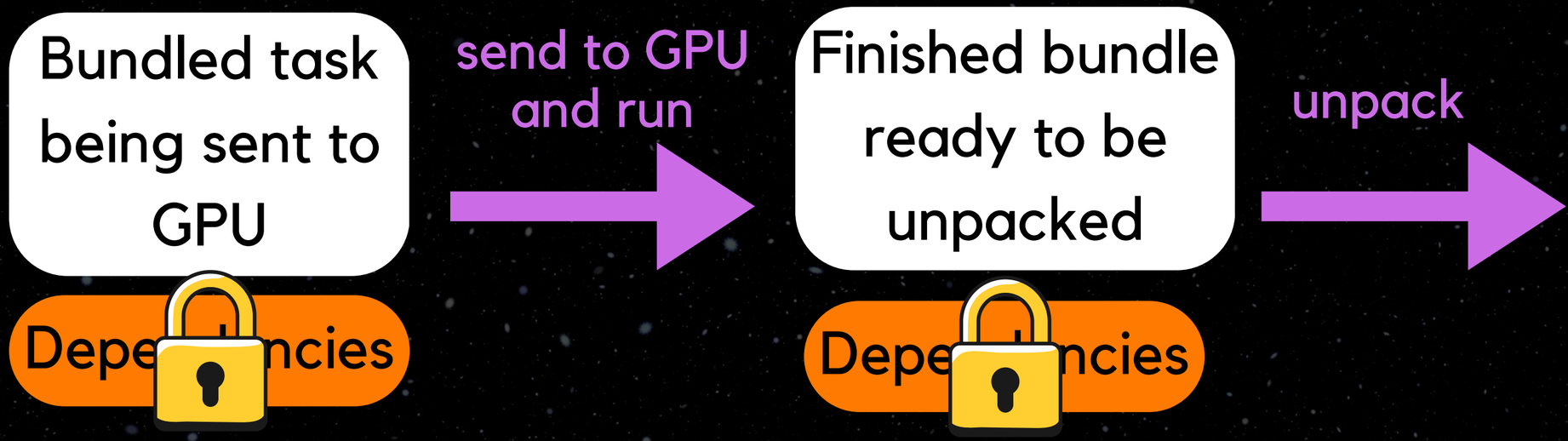
DEALING WITH TASKS

- Lots of task dependencies in SWIFT → need to ensure any tasks are still enqueued correctly afterwards



DEALING WITH TASKS

- Lots of task dependencies in SWIFT → need to ensure any tasks are still enqueued correctly afterwards

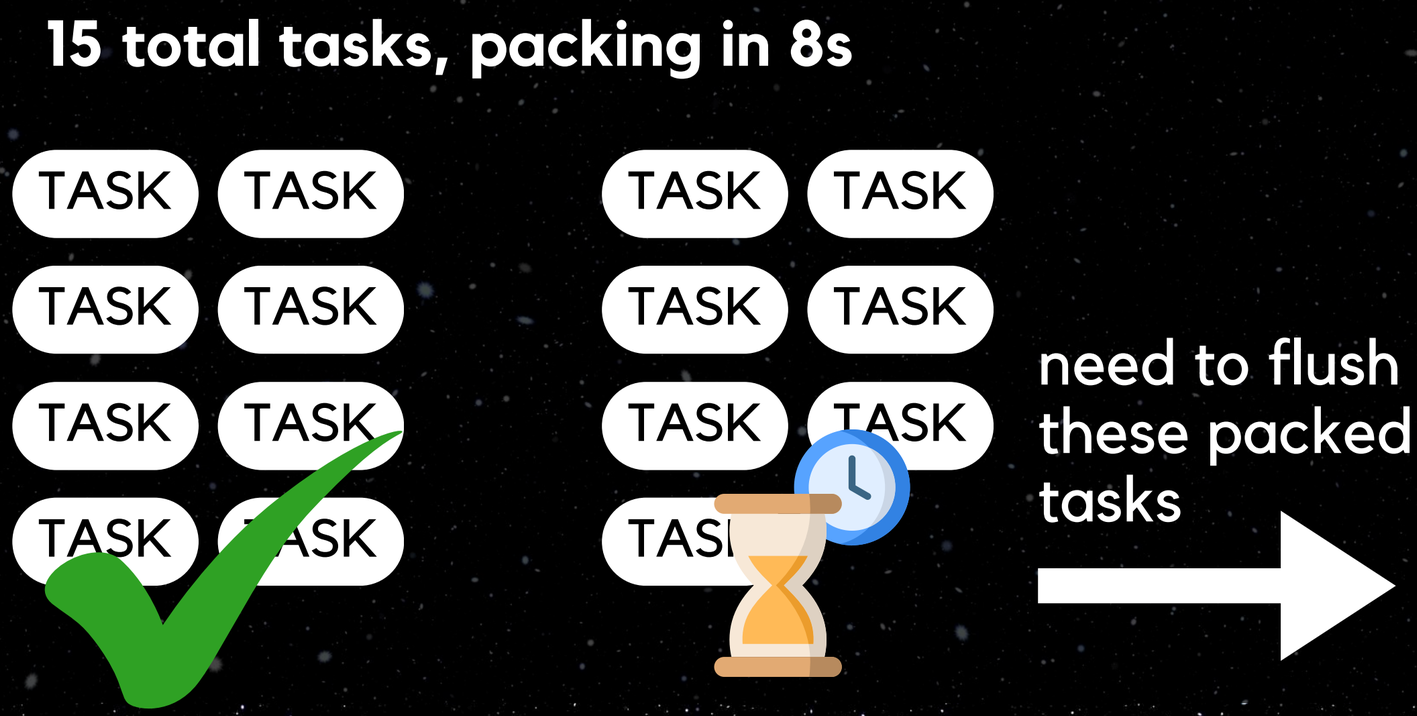


DEALING WITH TASKS

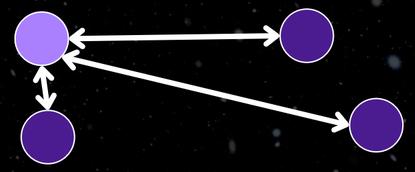
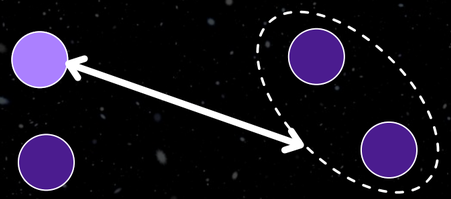
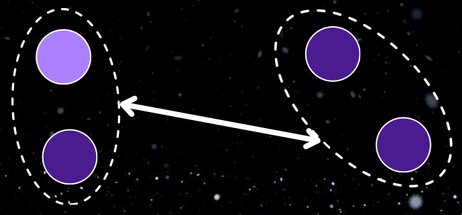
- Lots of task dependencies in SWIFT → need to ensure any tasks are still enqueued correctly afterwards

DEALING WITH TASKS

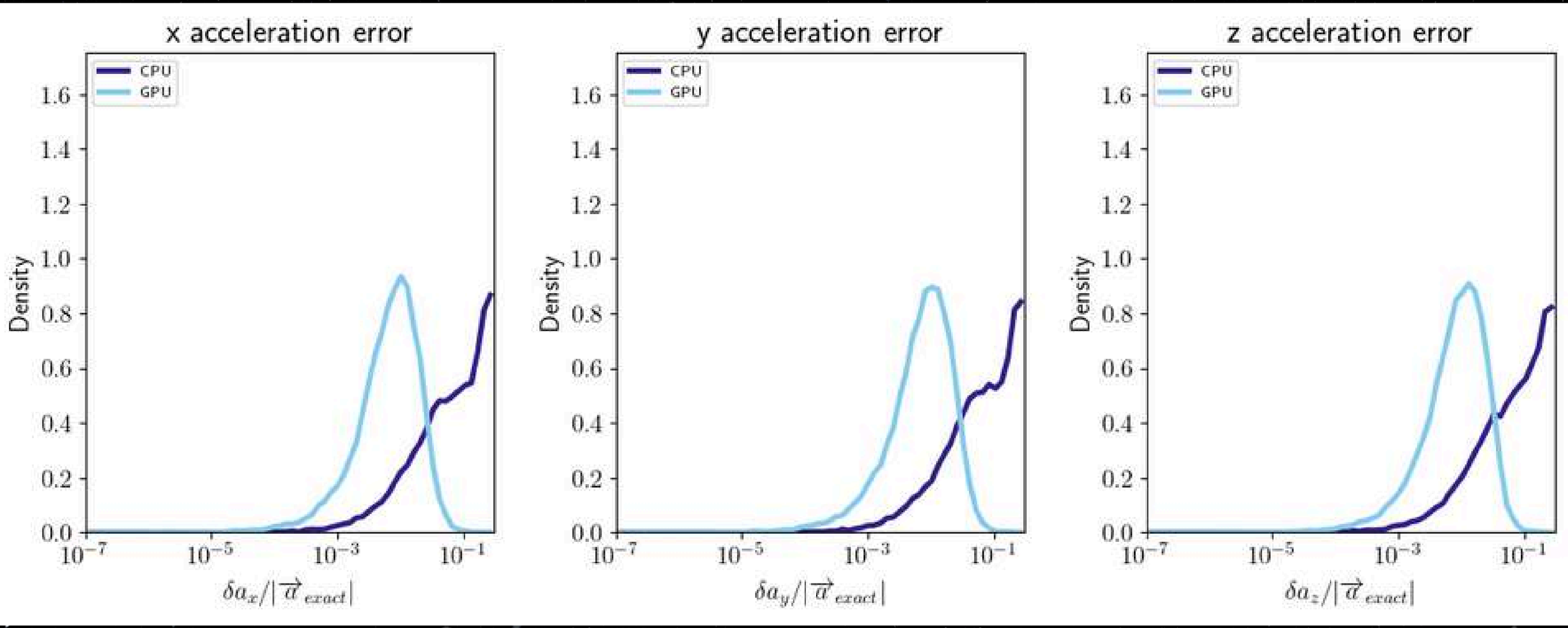
- Tasks leftover that don't make up a full pack → need to be flushed so code can proceed



GRAVITY RESTRUCTURE

CURRENT VERSION	INTERACTION	NEW GPU VERSION
On CPU	Particle-Particle 	**On GPU** Easy to parallelize on GPU
	Particle-Multipole 	**Change to P-P** P-P inexpensive on GPU and more accuate
	Multipole-Multipole 	**On CPU** Inexpensive on CPU and separate to other interactions

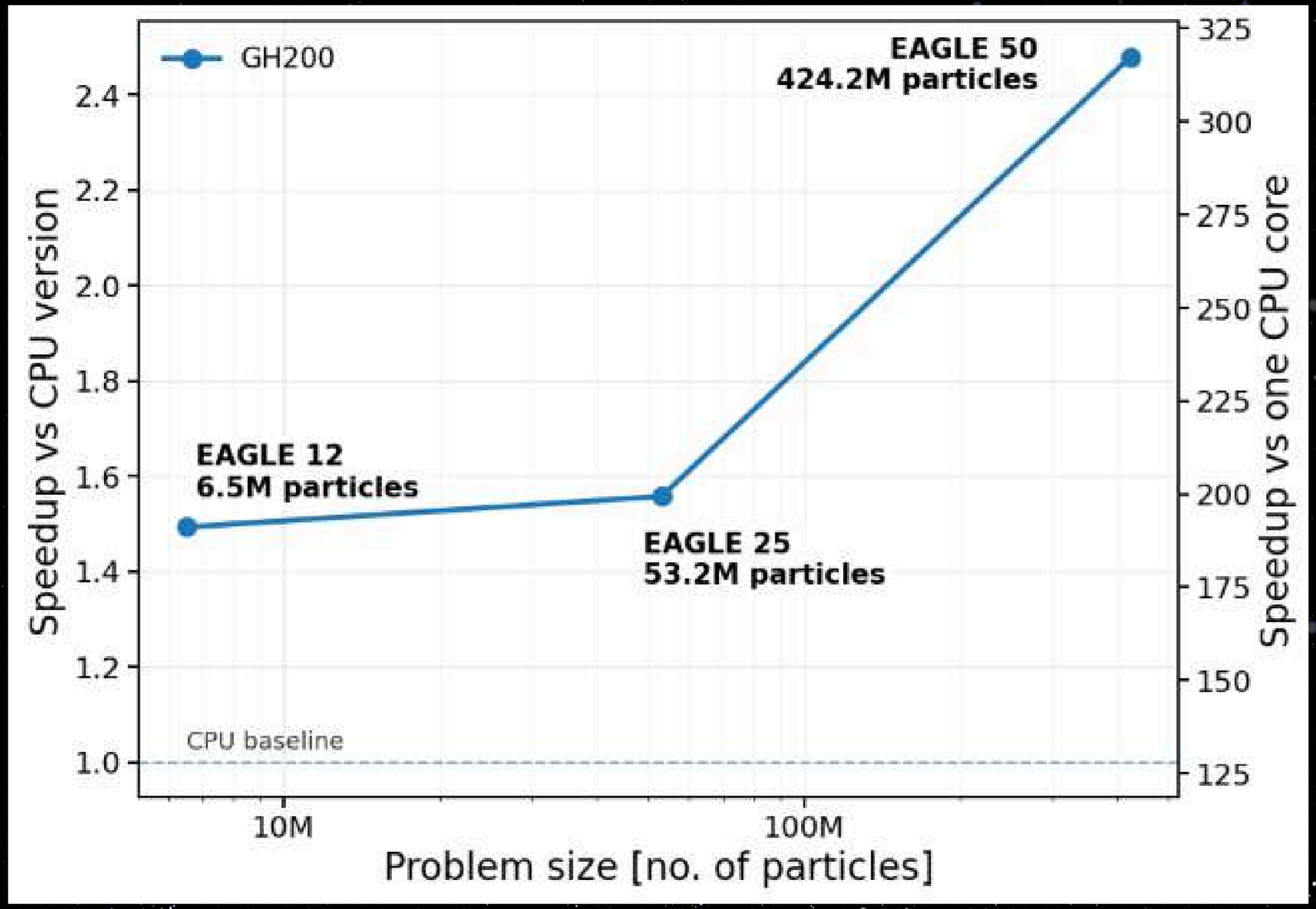
ACCURACY BENEFITS



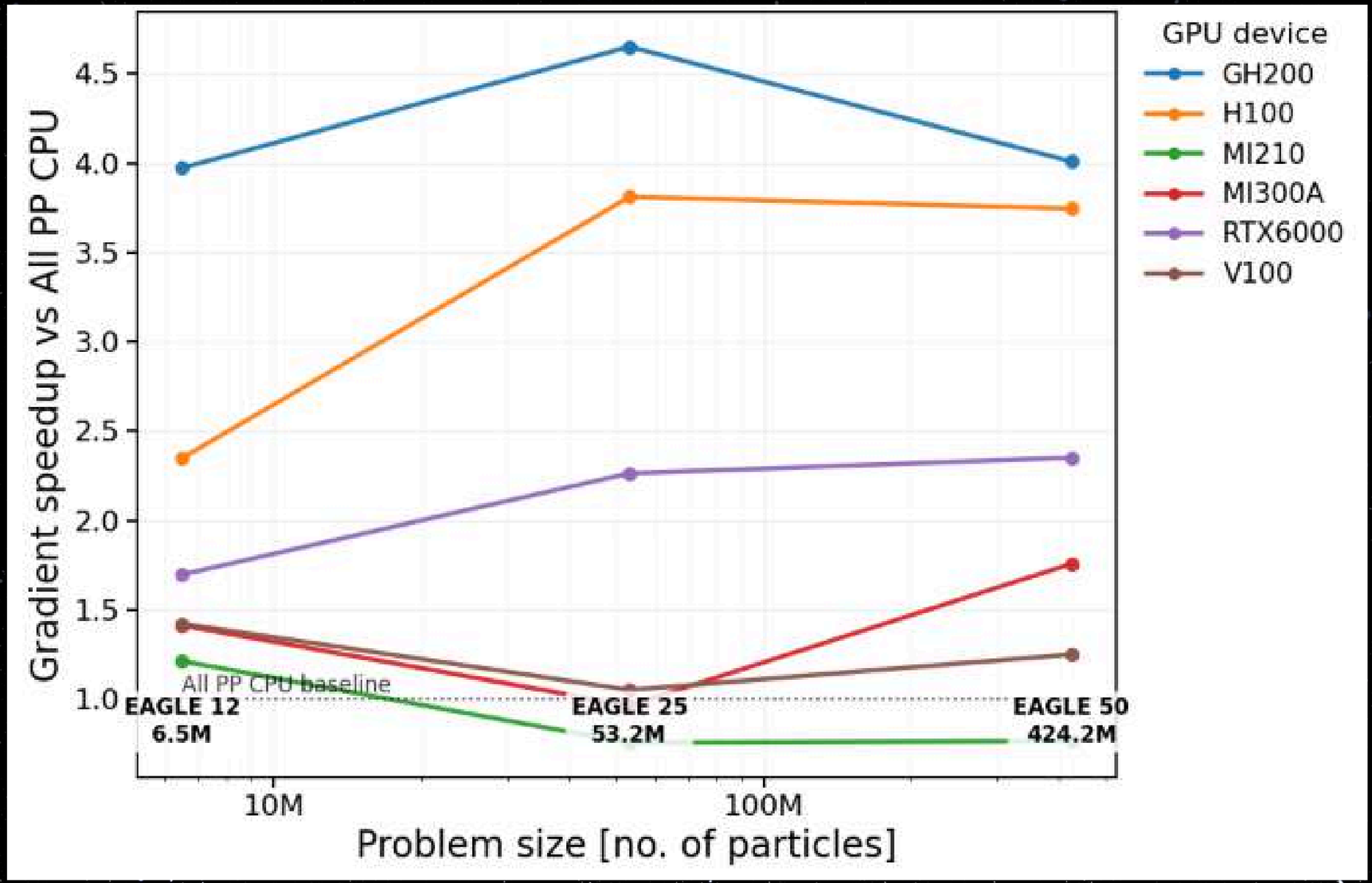
PORTABILITY

- Code uses an abstraction layer to allow easy compilation into CUDA or HIP
- Command line arguments to compile and link relevant libraries
- Automatic sizing commands (batch size, number of streams) to prevent memory overflows

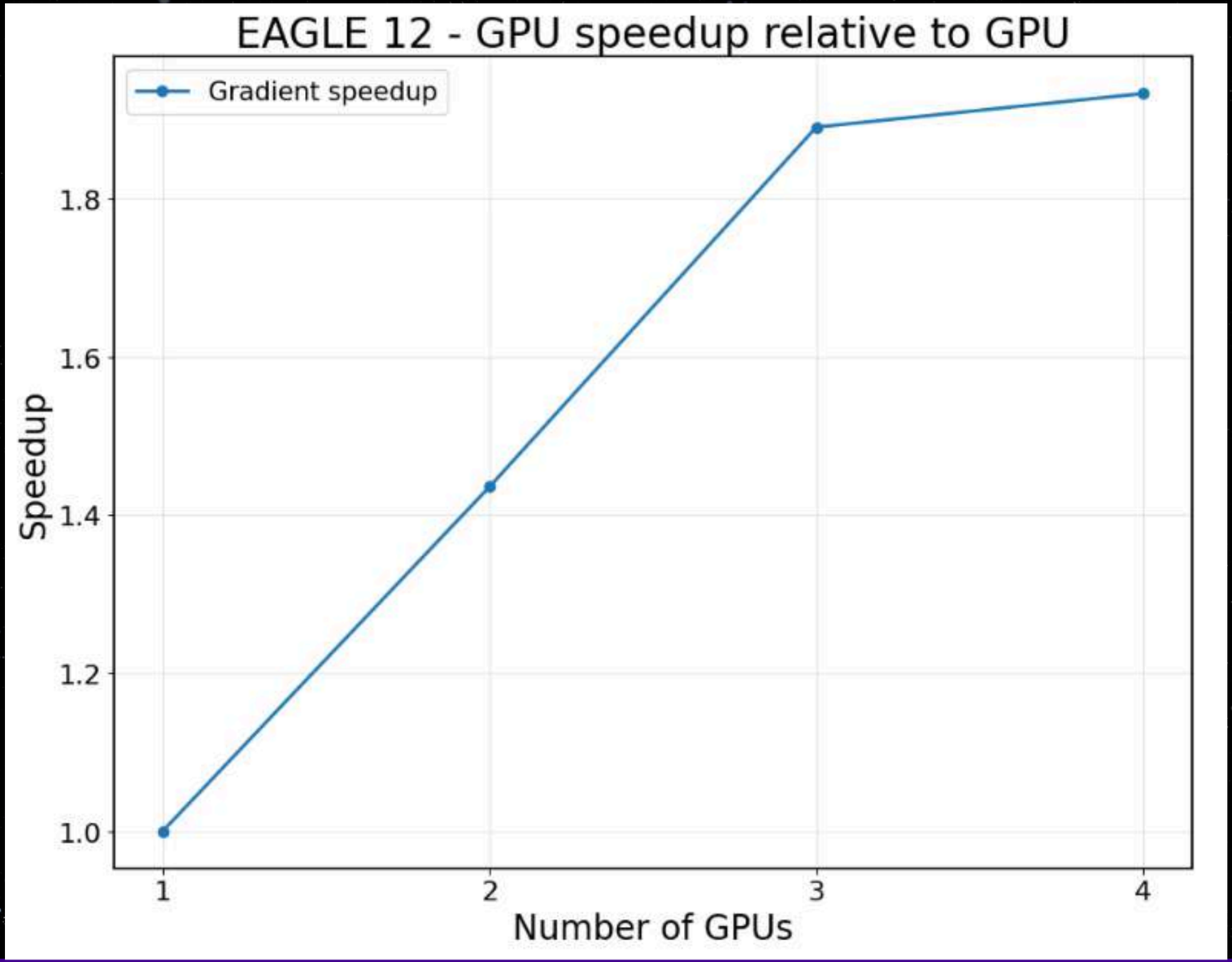
PERFORMANCE RESULTS



PERFORMANCE RESULTS



MULTI-GPU


EAGLE 12 - GPU speedup relative to GPU
Gradient speedup
Speedup
1.0
1.2
1.4
1.6
1.8
1
2
3
4
Number of GPUs

CONCLUSIONS

To adapt for future HPC systems SWIFT needs to utilise GPUs.

Initial results from the gravity GPU offload show higher accuracy results with correct science.

Single GPU: Speedup of ~4.5x for equivalent accuracy of calculations or ~2.5x compared to normal CPU version with approximations

Multi GPU: Exploring scaling with problem size.

sarah.c.johnston@durham.ac.uk sarah-c-johnston sarahdoesastrophysics